Experiment 3: Simulation of BPSK Modulation and Demodulation through an AWGN Channel

Output Code

```
import numpy as np
import matplotlib.pyplot as plt
import cv2
from scipy.special import erfc


img = cv2.imread("/content/cameraman.png", cv2.IMREAD_GRAYSCALE)

M, N = img.shape
print("Image size:", M, "x", N)

print(img.max())
print(img.min())
x = img / 255

print(x.max())
print(x.min())

x_vec = x.flatten()
```

PCM Encoder (8-bit Quantization)

```
def pcm_encode(x, b):
    L = 2**b
    delta = 1/L

    k = np.floor(x/delta)
    k = np.clip(k, 0, L-1).astype(int)
```

```
    xcap = (k + 0.5)*delta


    Ps = np.mean(x**2)
    Pn = np.mean((x - xcap)**2)
    SQNRdB = 10*np.log10(Ps/Pn)

    bitstream=""
    for v in k:
      bitstream+=np.binary_repr(v,width=b)

    b_tx = np.array(list(bitstream), dtype=np.uint8)


    return k, xcap, b_tx, SQNRdB
```

Channel Modelling – BPSK Modulation and AWGN

```
def bpsk_modulate(b):

    return 1 - 2*b.astype(complex)

def bpsk_demodulate(y):
    return (np.real(y) < 0).astype(np.uint8)

def channel_awgn_bpsk(b_tx, ebn0_db, rng=None):
    s = bpsk_modulate(b_tx)

    ebn0 = 10**(ebn0_db/10)
    N0 = 1/ebn0
    sigma = np.sqrt(N0)
    nI = np.random.normal(0, sigma, size=s.shape)
    nQ = np.random.normal(0, sigma, size=s.shape)

    n = (nI + 1j * nQ )/np.sqrt(2)
    y=s+n

    b_rx = bpsk_demodulate(y)
    ber = np.mean(b_rx != b_tx)

    return y,b_rx, ber
```

PCM Decoder and BER Computation

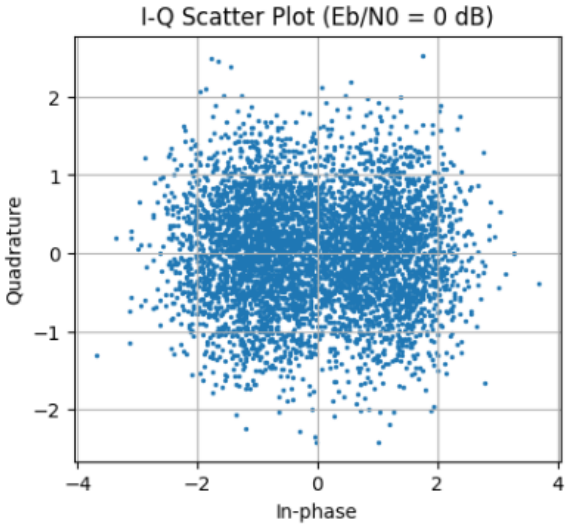
```
def pcm_decode(b_rx, b, M, N):
    L = 2**b
    delta = 1 / L

    total_bits = (len(b_rx) // b) * b
    b_rx = b_rx[:total_bits]

    bits = b_rx.reshape(-1, b)

    k_hat = []

    for row in bits:
        binary_string = ""
        for bit in row:
            binary_string += str(bit)

        index = int(binary_string, 2)
        k_hat.append(index)

    k_hat = np.array(k_hat)

    k_hat = np.clip(k_hat, 0, L-1)

    xcap = (k_hat + 0.5) * delta

    xcap_img = xcap.reshape(M, N)

    return xcap_img

def compute(x, x_hat):
    mse = np.mean((x - x_hat)**2)

    Ps = np.mean(x**2)
    Pn = mse
    snr_db = 10*np.log10(Ps/Pn)

    psnr_db = 10*np.log10(1/mse)
    return mse, snr_db, psnr_db

k, xcap, b_tx, SQNRdB = pcm_encode(x_vec, b=8)

print("Total transmitted bits =", len(b_tx))
print("SQNR (dB) =", SQNRdB)
```

Main Function

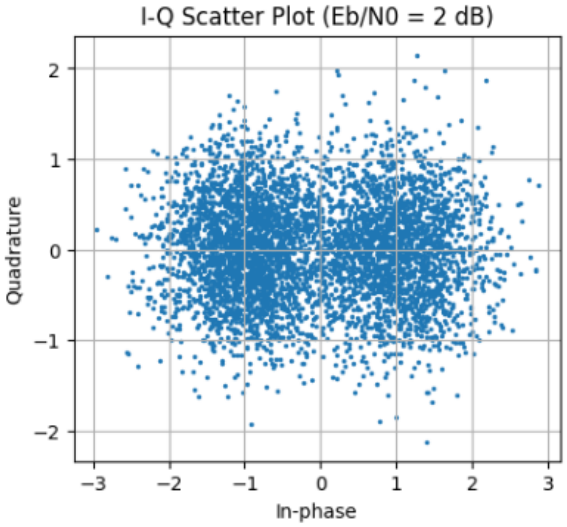
```
EbN0_dB = [0, 2, 4, 6, 8, 10]
BER_sim = []
BER_theory = []

for ebn0 in EbN0_dB:
    y, b_rx, ber = channel_awgn_bpsk(b_tx, ebn0)
    BER_sim.append(ber)

    xcap_img = pcm_decode(b_rx, b=8, M=M, N=N)

    plt.figure(figsize=(10,4))
    plt.subplot(1,2,1)
    plt.imshow(xcap_img, cmap='gray')
    plt.title(f"Eb/N0 = {ebn0} dB, BER = {ber:.3e}")
    plt.axis('off')


    plt.subplot(1,2,2)
    plt.scatter(np.real(y[:5000]), np.imag(y[:5000]), s=2)
    plt.xlabel("In-phase")
    plt.ylabel("Quadrature")
    plt.title(f"I-Q Scatter Plot (Eb/N0 = {ebn0} dB)")
    plt.grid(True)
    plt.show()


from scipy.special import erfc

BER_theory = [0.5 * erfc(np.sqrt(10**(e/10))) for e in EbN0_dB]

plt.semilogy(EbN0_dB, BER_sim, 'o-', label='Simulated')
plt.semilogy(EbN0_dB, BER_theory, '-', label='Theoretical')
plt.grid(True, which='both')
plt.legend()
plt.xlabel("Eb/N0 (dB)")
plt.ylabel("BER")
plt.show()
```

Reconstructed Images for Different Eb/N0 Values

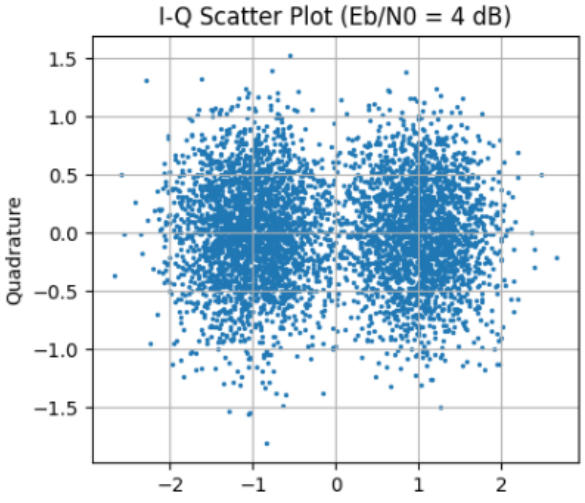
Eb/N0 = 0 dB, BER = 7.882e-02



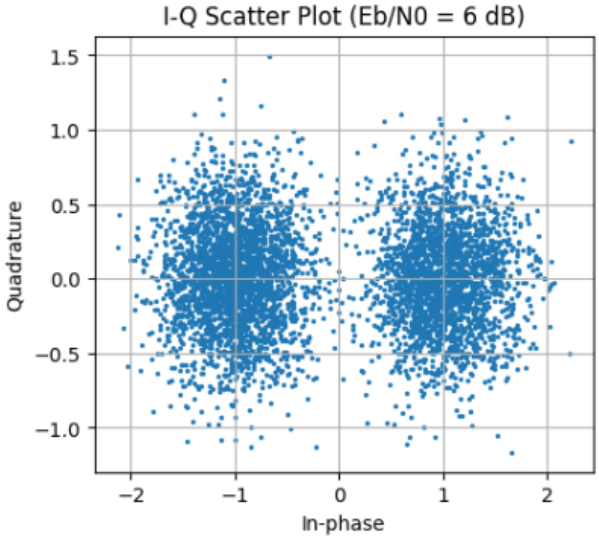
Eb/N0 = 2 dB, BER = 3.748e-02



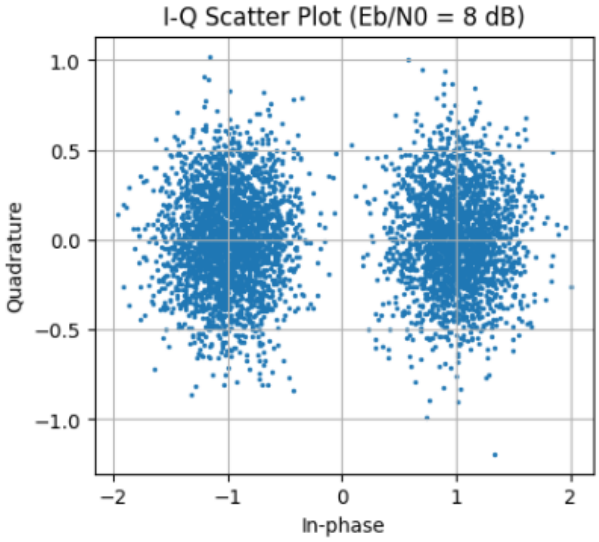
Eb/N0 = 4 dB, BER = 1.279e-02



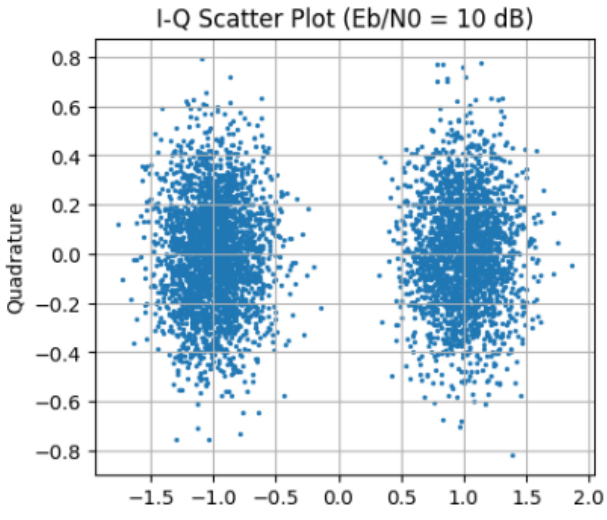
Eb/N0 = 6 dB, BER = 2.394e-03



Eb/N0 = 8 dB, BER = 2.003e-04



Eb/N0 = 10 dB, BER = 3.815e-06



BER vs Eb/N0 (Simulated vs Theoretical)